Facultad de Matemática y Computación
MatCom

Proyecto de IA

Hex Smart player

Autor:
Jocdan L. López Mantecón C-312

1. Introducción

HEX es un juego de estrategia de información perfecta para dos jugadores. Se juega sobre un tablero hexagonal de tamaño $N \times N$, donde cada casilla puede estar vacía o contener una ficha de uno de los dos jugadores. A diferencia de otros juegos de tablero más conocidos, en HEX no existe el empate: uno de los dos jugadores necesariamente logrará conectar sus lados objetivo si la partida se juega hasta el final. El jugador `1` busca unir el lado izquierdo con el derecho, mientras que el jugador `2` intenta conectar el borde superior con el inferior. Además, la representación del tablero representa como una matriz con valores `0`, `1` y `2`, y las adyacencias válidas se calculan usando el modelo `even-r`. Desde la perspectiva de la inteligencia artificial, HEX es especialmente interesante porque obliga a capturar la idea de conectividad, reconocer amenazas inmediatas, valorar posiciones intermedias y tomar decisiones dentro de un presupuesto de tiempo estricto.

2. Análisis preliminar

Al analizar qué enfoques estratégicos podrían emplearse para jugar HEX de manera competente, aparece en primer lugar una posibilidad evidente: usar una estrategia puramente **greedy**, es decir, evaluar todas las jugadas legales disponibles, asignarles una puntuación heurística según el estado resultante y escoger la mejor. Este enfoque tiene la ventaja de ser simple, rápido y fácil de implementar. Sin embargo, muchas jugadas solo revelan su verdadero valor varios turnos después, cuando cierran una ruta, preparan un puente o fuerzan una respuesta del rival.

Otra alternativa es el uso de **minimax** o **alpha-beta**, que permite modelar explícitamente el hecho de que el oponente también está jugando para optimizar su posición. Este tipo de búsqueda suele ser muy apropiado para juegos deterministas de dos jugadores, pero en HEX se enfrenta a un problema clásico: el factor de ramificación es alto, porque en casi cualquier momento hay muchas casillas vacías y explorar exhaustivamente el árbol se vuelve inviable salvo en fases muy tardías de la partida.

También se puede pensar en **Monte Carlo Tree Search** (MCTS), una familia de métodos muy respetada en juegos de tablero que combina exploración estadística con aprendizaje gradual del árbol de juego. No obstante, para este proyecto concreto, MCTS introduce una complejidad de implementación mayor y requiere *rollouts* o simulaciones que, si no están muy bien guiadas por conocimiento del dominio, pueden ser costosas o poco efectivas. Existen además enfoques más sofisticados, como heurísticas basadas en resistencia efectiva, patrones de conexiones virtuales o incluso redes neuronales, pero todos ellos elevan la complejidad conceptual y técnica.

Para este proyecto se concluye que conviene usar un enfoque híbrido: combinar chequeos tácticos directos, una buena heurística de conectividad y una búsqueda corta, selectiva y podada que capture algo de profundidad sin sacrificar robustez ni tiempo de respuesta.

3. Desarrollo

Si se observa con más detalle el desarrollo del juego, aparece con claridad una idea dominante: en HEX, el valor real de una jugada depende de cómo modifica la conectividad entre los bordes que cada jugador intenta unir. Esa observación conduce a las heurísticas basadas en caminos mínimos o *connection cost*, que fueron justamente las que se adoptaron como núcleo de la evaluación. La intuición es elegante y práctica: si una ficha propia ya ocupa una casilla, esa casilla tiene costo cero para la conexión; si la casilla está vacía, puede interpretarse como un paso con costo uno, porque aún haría falta ocuparla; y si está ocupada por el rival, puede tratarse como un obstáculo prácticamente infinito. Con esa asignación, el problema estratégico se traduce en encontrar el costo mínimo de conectar un borde objetivo con el opuesto, algo que puede resolverse eficientemente con una variante de Dijkstra.

Esta idea tiene varias ventajas. Primero, convierte la posición en una red de caminos factibles y bloqueos. Segundo, produce una magnitud interpretable: si mi costo de conexión disminuye y el del rival aumenta, la posición mejora para mí. Tercero, puede calcularse con rapidez razonable incluso varias veces por turno, siempre que se controle la profundidad de búsqueda y el número de jugadas candidatas. Cuarto, permite incorporar fácilmente el hecho de que los dos jugadores tienen objetivos geométricos diferentes: uno conecta horizontalmente y el otro verticalmente. En el código, esta idea se materializa principalmente en la función `_connection_cost`, que construye distancias sobre la matriz y usa una cola de prioridad para propagar el mejor costo acumulado hasta el borde objetivo.

Sin embargo, conviene complementarla con heurísticas locales que ayuden a seleccionar mejor las jugadas y a capturar patrones tácticos que el costo de camino no siempre refleja de inmediato. Una de las más útiles es la prioridad por amenazas y oportunidades inmediatas: si existe una jugada que gana en este turno, debe jugarse sin más análisis; y si el oponente tiene una jugada ganadora inmediata, lo correcto es bloquearla antes de pensar en planes de largo plazo. En el código, esta lógica aparece de forma muy directa en `_find_immediate_win`, que prueba cada casilla vacía para verificar si al colocarla se completa una conexión válida.

Otra heurística local importante es la cercanía a fichas propias y rivales. Una casilla vecina a fichas propias suele servir para consolidar un grupo o extender una cadena; una vecina a fichas rivales puede resultar crucial para bloquear o disputar un corredor de conexión. Por eso, la función `_generate_candidates` otorga puntuaciones a las casillas en función de cuántos vecinos propios u oponentes tienen, añadiendo además una bonificación cuando la misma casilla toca simultáneamente presencia de ambos jugadores, ya que suele tratarse de un punto de tensión táctica. También se incorpora una preferencia por zonas centrales mediante una penalización por distancia al centro, no porque el centro sea siempre la mejor jugada, sino porque en fases iniciales y medias del juego las casillas centrales suelen ofrecer mayor flexibilidad estructural.

Finalmente, la función `_cell_corridor_bonus` intenta capturar otra idea útil: no todas las casillas valen lo mismo para el mismo jugador, y es razonable premiar aquellas que se ubican de manera favorable respecto a los lados objetivo y que además tienen apoyo amistoso cercano. En conjunto, estas heurísticas no reemplazan a la evaluación global por conectividad, sino que la estabilizan y hacen que el motor examine primero las jugadas con mayor relevancia estructural.

Una vez definidas las señales heurísticas, surge la cuestión de cómo convertirlas en decisiones concretas frente a un oponente inteligente. Aquí es donde entra la búsqueda `alpha-beta` adoptada en la solución. La razón de elegir `alpha-beta`, en lugar de una búsqueda exhaustiva o un enfoque puramente codicioso, es que introduce un equilibrio razonable entre profundidad estratégica y eficiencia. El método parte de la idea minimax clásica: cuando me toca jugar, quiero maximizar mi evaluación; cuando juega el rival, debo asumir que intentará minimizarla. La mejora de `alpha-beta` consiste en descartar ramas del árbol que ya no pueden alterar la decisión final, lo que permite explorar más con el mismo presupuesto de tiempo.

En el código, esta lógica se implementa en `_alphabeta`, donde la búsqueda alterna entre el jugador actual y su oponente, corta cuando se agota la profundidad o el tiempo y devuelve valores extremos si detecta una conexión ganadora para alguno de los bandos. Un aspecto especialmente importante en esta implementación es que la búsqueda se hace siempre desde la perspectiva del jugador raíz: la evaluación `_evaluate` se calcula respecto al jugador que originalmente llamó a `play`, y no se mezclan signos de forma confusa. Por otra parte, la búsqueda no se aplica sobre todas las jugadas legales, sino sobre un conjunto reducido de candidatos ya preseleccionados por `_generate_candidates`.

En términos de estructura del código, las partes principales pueden resumirse así:

- La función `play` organiza toda la lógica de alto nivel: copia la matriz, detecta si el tablero está vacío para elegir una apertura centrada, busca victorias inmediatas propias, busca bloqueos urgentes frente al rival, fija un límite de tiempo interno y decide la profundidad de búsqueda y el número máximo de candidatos según cuántas casillas vacías queden.
- La función `_neighbors` codifica exactamente las seis direcciones válidas del sistema `even-r`.
- La función `_has_connection` verifica si un jugador ya unió sus lados mediante un recorrido en profundidad, y es utilizada tanto para detectar victorias inmediatas como para cortar la búsqueda con un resultado terminal.
- `_connection_cost` aporta la señal global principal de la evaluación.
- `_generate_candidates` implementa la política selectiva de exploración, que es uno de los elementos más decisivos para el rendimiento real del agente.
- `_evaluate` fusiona varias señales en una puntuación única, equilibrando costo de conexión, adyacencia amistosa y presencia central.
- Otras funciones auxiliares como `_find_immediate_win` o `_cell_corridor_bonus` encapsulan ideas tácticas específicas que enriquecen la toma de decisiones sin complicar la lógica principal.

4. Conclusiones

El jugador resultante combina una apertura razonable cerca del centro, chequeos tácticos inmediatos para no dejar pasar victorias obvias ni permitir amenazas de un turno, una evaluación global basada en costo mínimo de conexión, heurísticas locales de vecindad y corredor para ordenar y filtrar movimientos, y una búsqueda `alpha-beta` corta que añade profundidad sin comprometer el tiempo de ejecución. Las estrategias y heurísticas que adopta son coherentes entre sí: primero asegurar la corrección táctica, luego estimar la conectividad global, después refinar la relevancia local de las jugadas y finalmente usar una búsqueda selectiva para anticipar la respuesta del oponente.